GPillT: An Object-Oriented Medical QA System for Elderly Polypharmacy Medication Safety

202212464 Jo Eun-Yeong
Object-Oriented Programming

1. Introduction

This project implements an object-oriented system related to my future interest in healthcare AI, medical data systems, and drug safety support. The system is called **GPillT**, which stands for **General Pill Toolkit**. GPillT is a simplified medical question-answering system designed for an elderly polypharmacy scenario. Polypharmacy refers to a situation where a patient takes multiple medications at the same time. In older adults, this situation is common because one patient may manage several chronic diseases and take several prescriptions or over-the-counter drugs daily.

The motivation for choosing this topic comes from the importance of medication safety. When multiple drugs are taken together, there may be risks such as drug-drug interactions, duplicate active ingredients, contraindicated combinations, or confusing dosage information. For elderly patients, these risks can become more serious because they often have complex medication histories. Therefore, a system that can represent patients, drugs, drug information, interaction rules, and user questions in a structured way is meaningful as an object-oriented programming project.

My future career interest is closely related to healthcare AI and biomedical AI systems. In this field, it is not enough for a program to simply produce an answer. The system should also be organized in a way that is reliable, explainable, and easy to extend. For example, a medication QA system may later need to support more drugs, more patient information, additional question types, or integration with a real drug database. Object-oriented programming is useful for this kind of system because it allows real-world entities and responsibilities to be separated into classes.

The objective of this project is to practically implement a small but complete object-oriented medical QA system. The system receives natural-language-style questions, analyzes the question intent, retrieves sample drug information from an in-memory database, checks simplified medication safety rules, and generates readable responses. The purpose is not to provide real medical advice. All drug information, interaction rules, and recommendations are educational sample data only.

The main goals of this project are as follows:

- to model healthcare-related entities using object-oriented classes;
- to separate the system into modules such as database search, question analysis, interaction checking, and response generation;
- to demonstrate inheritance and polymorphism through question analyzer classes;
- to show how an object-oriented structure can support future expansion of a healthcare AI system.

The final source code is available at:

`https://github.com/tina1lure/gpillt-oop-assignment`

2. Related Work

GPillT is inspired by three types of existing systems or concepts: medication information systems, drug interaction checkers, and medical question-answering systems.

Medication information systems provide structured information about drugs. A typical drug record may include the drug name, ingredients, possible side effects, dosage information, warnings, and contraindications. This kind of data is naturally suitable for object-oriented modeling. For example, each medication can be represented as a `Drug` object, and the object's attributes can store the drug's ingredients, side effects, and dosage. This is one reason why the drug safety domain is appropriate for an OOP project.

Drug interaction checkers compare two or more medications and identify possible safety risks. In real systems, this process requires clinical databases and expert-validated rules. In this project, the interaction checking logic is intentionally simplified. GPillT only checks whether one drug's contraindicated ingredient appears in another drug's ingredient list, or whether two drugs share duplicate ingredients. Although the logic is simple, the structure is similar to a real medication safety workflow: retrieve drug information, compare drug properties, and return a structured safety result.

Medical question-answering systems and healthcare chatbots attempt to answer user questions written in natural language. A user may ask about drug interactions, side effects, dosage, or general medication guidance. In a real AI system, this may require natural language processing, retrieval-augmented generation, or a large language model. However, this project keeps the question analysis keyword-based so that the implementation remains understandable and focused on object-oriented design.

Compared with these related systems, GPillT is much smaller and educational. It does not use real clinical databases, real patient records, or a large language model. However, it captures the core structure of a medication QA system in an object-oriented way. It contains separate objects for drugs, patients, databases, analyzers, checkers, safety results, and response generation. Therefore, the project demonstrates how OOP can be used to design the skeleton of a healthcare AI system.

The key difference between GPillT and a simple procedural script is the design philosophy. A procedural script might store all logic in one long function, using many conditional statements. GPillT instead divides the workflow into responsible objects. This makes the program easier to understand, debug, and extend. For example, adding a new question type does not require rewriting the whole program. A new analyzer class can be added while keeping the existing classes mostly unchanged.

3. Proposed OOP System

3.1. Overall System Architecture

The proposed system follows a pipeline architecture. Each stage of the pipeline is handled by a different class or module.

```
User Question
    |
    v
Question Analyzer
    |
    v
Drug Database
    |
    v
Interaction Checker
    |
    v
Response Generator
    |
    v
Final Response
```

The system workflow can be summarized as follows. First, the user provides a medication-related question. Second, the question analyzer checks the question intent and extracts drug names. Third, the drug database retrieves matching `Drug` objects. Fourth, if the question is about drug interaction, the `InteractionChecker` compares the selected drugs and creates a `SafetyResult`. Finally, the `ResponseGenerator` converts the result into a readable answer.

Stage	Responsibility
Question Analyzer	Classifies the question intent and extracts related drug names.
Drug Database	Stores sample drug objects and returns matching drug records.
Interaction Checker	Compares two drugs and produces a structured safety result.
Response Generator	Converts system results into readable terminal responses.

3.2. File Structure

The project is divided into multiple files so that each file has a clear responsibility.

```
gpillt-oop-assignment/
|
|-- README.md
|-- src/
    |-- main.py
    |-- models.py
```

```
|-- database.py
|-- analyzers.py
|-- checker.py
|-- response.py
```

`models.py` defines the core domain objects. `database.py` manages sample drug data. `analyzers.py` contains the question analyzer classes. `checker.py` implements medication interaction checking. `response.py` formats the final answers. `main.py` connects all components and runs the demonstration.

3.3. Class Design

The main classes of GPillT are listed below.

Class	Role
`Drug`	Represents a medication with name, ingredients, side effects, dosage information, and contraindicated ingredients.
`Patient`	Represents a patient with name, age, and current medication list.
`SafetyResult`	Stores the result of a medication safety check, including status, reason, and recommendation.
`DrugDatabase`	Stores sample drug records and searches for drugs by name.
`InteractionChecker`	Checks whether two drugs have a dangerous or caution-level interaction under simplified rules.
`BaseQuestionAnalyzer`	Abstract parent class defining the common interface for all question analyzers.
`InteractionAnalyzer`	Child analyzer class for drug interaction questions.
`SideEffectAnalyzer`	Child analyzer class for side-effect questions.
`DosageAnalyzer`	Child analyzer class for dosage questions.
`ResponseGenerator`	Formats interaction, side-effect, and dosage results for output.

This class design follows the principle of separation of responsibilities. The `Drug` class only stores medication-related information. The `Patient` class manages the patient's medication list. The analyzer classes analyze questions, while the checker class performs medication safety checking. The response generator only handles output formatting. Because each class has a focused responsibility, the system is easier to read and maintain.

3.4. Domain Model Implementation

The core domain objects are defined in `models.py`. The `Drug` class stores medication information, and the `Patient` class manages a patient's current drugs. The `SafetyResult` class stores the structured output of an interaction check.

Listing 1: Simplified domain model design

```
class Drug:
   def __init__(self, name, ingredients, side_effects,
              dosage_info, contraindicated_ingredients):
      self.name = name
      self.ingredients = ingredients
      self.side_effects = side_effects
      self.dosage_info = dosage_info
      self.contraindicated_ingredients = contraindicated_ingredients


class Patient:
   def __init__(self, name, age):
      self.name = name
      self.age = age
      self.current_drugs = []

   def add_drug(self, drug):
      self.current_drugs.append(drug)


class SafetyResult:
   def __init__(self, status, reason, recommendation):
      self.status = status
      self.reason = reason
      self.recommendation = recommendation
```

This design shows encapsulation because related data is grouped inside meaningful classes. For example, medication attributes are stored inside a `Drug` object, and patient-specific medication history is stored inside a `Patient` object.

3.5. Database and Interaction Checking

The `DrugDatabase` class stores four sample drugs: `Norvasc`, `Warfarin`, `Aspirin`, and `VitaminC`. It acts as the single source of drug information in the system.

Listing 2: Simplified database responsibility

```
class DrugDatabase:
   def search_drug(self, name):
      # Returns a Drug object that matches the given name
      pass

   def get_all_drugs(self):
      # Returns all sample Drug objects
      pass
```

The `InteractionChecker` compares two `Drug` objects and returns a `SafetyResult`. The checking logic is deliberately simple for the assignment. If one drug's contraindicated ingredient appears in the other drug's ingredients, the system returns `DANGER`. If two drugs share ingredients, the system returns `CAUTION`. Otherwise, the system returns `SAFE`.

Listing 3: Simplified interaction checking logic

```
class InteractionChecker:
   def check_interaction(self, drug1, drug2):
      danger = self.check_contraindications(drug1, drug2)
      if danger:
         return SafetyResult("DANGER", danger,
                        "Avoid taking these together in the demo.")

      duplicate = self.check_duplicate_ingredients(drug1, drug2)
      if duplicate:
         return SafetyResult("CAUTION", duplicate,
                        "Check duplicate ingredients.")

      return SafetyResult("SAFE", "No sample rule triggered.",
                     "No sample interaction found.")
```

This structure is important because the safety decision is not mixed with the question analysis or output formatting. The checker has one main responsibility: compare drugs and return a safety result.

3.6. Question Analyzer Design

The analyzer classes are the main example of inheritance and polymorphism in this system. The abstract parent class `BaseQuestionAnalyzer` defines the common interface `analyze(question)`. Each child class implements the same method differently.

Listing 4: Inheritance and polymorphism in analyzers

```
from abc import ABC, abstractmethod

class BaseQuestionAnalyzer(ABC):
   @abstractmethod
   def analyze(self, question):
      pass


class InteractionAnalyzer(BaseQuestionAnalyzer):
   def analyze(self, question):
      # Detects interaction-related questions
      pass


class SideEffectAnalyzer(BaseQuestionAnalyzer):
   def analyze(self, question):
      # Detects side-effect-related questions
      pass


class DosageAnalyzer(BaseQuestionAnalyzer):
   def analyze(self, question):
```

```
        # Detects dosage-related questions
        pass
```

In `main.py`, the analyzers are stored in a single list. The system calls the same method, `analyze(question)`, on every analyzer object.

Listing 5: Polymorphic analyzer loop

```
analyzers = [
    InteractionAnalyzer(database),
    SideEffectAnalyzer(database),
    DosageAnalyzer(database),
]

for analyzer in analyzers:
    result = analyzer.analyze(question)
    print(type(analyzer).__name__, "->", result)
```

This is polymorphism. The caller uses the same method name, but each analyzer behaves differently depending on its class. This makes the design extensible. If a new question type is added later, a new analyzer class can inherit from `BaseQuestionAnalyzer` and implement its own `analyze()` method.

4. Implementation & Usage / Results

4.1. Implementation Environment

The project was implemented in Python using only the standard library. No external APIs, real medical databases, or real patient records were used. The program can be executed from the project root directory with the following command:

```
python src/main.py
```

The program prints a terminal-based demonstration. It first shows a patient medication demo and then answers three sample questions. The output is designed to be readable enough for a report screenshot or execution log.

4.2. Actual Execution Output

The actual execution result is shown below.

Listing 6: Actual output of python src/main.py

```
python src/main.py
======================================================================
GPillT: Inheritance and Polymorphism Demo
Educational sample only -- NOT real medical advice
======================================================================

======================================================================
Patient Medication Demo (Elderly Polypharmacy)
----------------------------------------------------------------------
Patient: Elderly Patient, Age: 75
Current Drugs: ['Warfarin', 'Aspirin']

Sample interaction check for this patient's medications:
[Interaction Check]
  Status: DANGER
  Reason: Warfarin lists contraindicated ingredient 'salicylate' which appears in Aspirin (sample rule)
      .
  Recommendation: Demo: avoid taking these together. Consult a healthcare professional in real life.

======================================================================
QUESTION: Can I take Warfarin together with Aspirin?
----------------------------------------------------------------------
  Polymorphism -- each analyzer.analyze(question):
    InteractionAnalyzer -> {'intent': 'interaction', 'drug_names': ['Warfarin', 'Aspirin']}
    SideEffectAnalyzer -> {'intent': '', 'drug_names': []}
    DosageAnalyzer -> {'intent': '', 'drug_names': []}
----------------------------------------------------------------------
ANSWER:
[Interaction Check]
  Status: DANGER
```

```
  Reason: Warfarin lists contraindicated ingredient 'salicylate' which appears in Aspirin (sample rule)
      .
  Recommendation: Demo: avoid taking these together. Consult a healthcare professional in real life.

======================================================================
QUESTION: What are the side effects of Norvasc?
----------------------------------------------------------------------
  Polymorphism -- each analyzer.analyze(question):
    InteractionAnalyzer -> {'intent': '', 'drug_names': []}
    SideEffectAnalyzer -> {'intent': 'side_effect', 'drug_names': ['Norvasc']}
    DosageAnalyzer -> {'intent': '', 'drug_names': []}
----------------------------------------------------------------------
ANSWER:
[Side Effects]
  Drug: Norvasc
  Sample side effects: dizziness (sample), swelling (sample)
  Note: Educational sample data only -- not real medical advice.

======================================================================
QUESTION: What is the dosage of VitaminC?
----------------------------------------------------------------------
  Polymorphism -- each analyzer.analyze(question):
    InteractionAnalyzer -> {'intent': '', 'drug_names': []}
    SideEffectAnalyzer -> {'intent': '', 'drug_names': []}
    DosageAnalyzer -> {'intent': 'dosage', 'drug_names': ['VitaminC']}
----------------------------------------------------------------------
ANSWER:
[Dosage Info]
  Drug: VitaminC
  Sample dosage: 500 mg once daily (sample only)
  Note: Educational sample data only -- not real medical advice.

======================================================================
Demo complete.
======================================================================
```

4.3. Result Interpretation

The first part of the output shows the elderly polypharmacy scenario. A `Patient` object is created with the name `Elderly Patient` and age 75. Then, `Warfarin` and `Aspirin` are retrieved from the database and added to the patient's current medication list. The system prints the patient's current drugs as `['Warfarin', 'Aspirin']`.

The interaction checker then checks the two medications. The result is `DANGER`. In the sample rule, `Warfarin` lists `salicylate` as a contraindicated ingredient, and `Aspirin` includes `salicylate`. Therefore, the system prints a warning message. This demonstrates how `Patient`, `DrugDatabase`, `Drug`, `InteractionChecker`, and `SafetyResult` cooperate in one workflow.

The second part of the output shows the question-answering demo. For the question "Can I take Warfarin together with Aspirin?", the program calls `analyze(question)` on every analyzer. Only `InteractionAnalyzer` returns the intent `interaction` and extracts the two drug names. The other analyzers return empty results. This confirms that the analyzer classes share the same interface but behave differently.

For the question "What are the side effects of Norvasc?", only `SideEffectAnalyzer` returns the intent `side_effect`. The response generator then prints the sample side effects of `Norvasc`. For the question "What is the dosage of VitaminC?", only `DosageAnalyzer` returns the intent `dosage`, and the response generator prints the sample dosage information.

Overall, the output confirms that the object-oriented pipeline works correctly. The system does not simply print fixed answers. Instead, it analyzes the question type, retrieves drug information, applies interaction-checking logic when needed, and generates an appropriate response.

5. Limitations

Although GPillT demonstrates an object-oriented system successfully, it has several limitations.

First, the drug database is very small. It contains only four sample drugs: `Norvasc`, `Warfarin`, `Aspirin`, and `VitaminC`. A real medication safety system would require a much larger and clinically validated drug database.

Second, the medication safety logic is highly simplified. Real drug-drug interactions cannot be determined only by matching ingredients. They may depend on dose, frequency, patient age, kidney function, liver function, disease history, laboratory values, and other medications. GPillT does not model these clinical factors.

Third, the question analysis is keyword-based. It detects simple words such as `together`, `side effects`, and `dosage`. This is enough for an OOP assignment, but it is not robust for real natural language questions. A future version could use natural language processing or a large language model to classify intent more accurately.

Fourth, the system runs only in the terminal. It does not provide a graphical user interface, mobile application, user authentication, persistent database, or caregiver mode. Since the target scenario is elderly polypharmacy, a practical system would need a simple interface that older adults or caregivers can use easily.

Fifth, the current project does not include expert verification. A real medication QA system should involve pharmacists, physicians, or validated medical knowledge sources. GPillT is only an educational demo and should not be used for real medical decisions.

For future improvements, the system could be extended in several directions. It could connect to a real drug database, support more medication safety rules, store patient medication history persistently, use NLP or LLM-based question analysis, and provide a web or mobile interface. It could also include a pharmacist review step before giving final medication safety recommendations.

6. Conclusion

This project implemented GPillT, an object-oriented medical QA system for elderly polypharmacy medication safety. The system was designed around my future interest in healthcare AI and drug safety support systems. Instead of building a real clinical product, I focused on creating a complete and explainable OOP architecture.

The project modeled healthcare concepts using classes such as `Patient`, `Drug`, `SafetyResult`, `DrugDatabase`, `InteractionChecker`, and `ResponseGenerator`. It also used inheritance and polymorphism through `BaseQuestionAnalyz` `InteractionAnalyzer`, `SideEffectAnalyzer`, and `DosageAnalyzer`. This design allowed different question types to be processed through the same interface while keeping each analyzer's behavior separate.

Through the implementation and execution results, I confirmed that object-oriented programming is useful for organizing real-world systems into clear, maintainable components. In the GPillT example, each class corresponds to a meaningful responsibility in a medication QA workflow. This makes the system easier to extend, test, and explain.

This project also helped me understand how OOP can support future healthcare AI systems. Even though the current version is simple, the architecture can be expanded with more drugs, more patient information, real databases, better question analysis, and safer response generation. Therefore, GPillT demonstrates not only OOP syntax, but also the value of object-oriented design for building structured and extensible healthcare software.